

Sistemas de Informação Gerencial e Bancos de Dados: Fundamentação Teórica e Procedimento Operacional Padrão para Acesso e Extração de Dados com PostgreSQL e MongoDB

Petter de Oliveira Braga Attux

Universidade Federal de Goiás, Faculdade de Ciências e Tecnologia

Câmpus Aparecida de Goiânia, Curso de Engenharia de Produção

petter.attux@discente.ufg.br

Resumo: Este artigo apresenta a fundamentação teórica e o Procedimento Operacional Padrão (POP) para acesso, extração e análise de dados em sistemas de gestão da produção. São abordados os conceitos de Sistemas de Informação Gerencial (SIG), arquitetura de software em camadas e o universo dos bancos de dados: histórico, modelo relacional, tipos NoSQL (documento, chave-valor, colunar e grafo) e critérios de escolha entre paradigmas. São detalhados ainda a normalização até a Terceira Forma Normal (3FN), os tipos de relacionamentos entre entidades e os procedimentos de conexão aos dois SGBDs candidatos para a parte prática: PostgreSQL e MongoDB. O objetivo é demonstrar como o ciclo dado, informação e conhecimento pode ser operacionalizado pelo Engenheiro de Produção para embasar decisões estratégicas e operacionais.

Palavras-chave: Sistema de Informação Gerencial; Bancos de Dados; PostgreSQL; MongoDB; NoSQL; Normalização; Engenharia de Produção.

1. INTRODUÇÃO

A crescente complexidade dos ambientes industriais exige que profissionais da Engenharia de Produção sejam capazes de acessar, interpretar e transformar dados brutos em informações úteis para a tomada de decisão. Nesse contexto, os Sistemas de Informação Gerencial (SIG) emergem como infraestrutura crítica para a gestão eficiente dos recursos produtivos (LAUDON; LAUDON, 2014).

Segundo Oliveira (2018), a ausência de informação de qualidade resulta em decisões fundamentadas em intuição, estoques desbalanceados e perda de visibilidade sobre o processo produtivo. O banco de dados é o alicerce tecnológico de qualquer SIG, e compreender o universo de opções disponíveis, dos sistemas relacionais clássicos às modernas soluções NoSQL, é condição necessária para que o engenheiro selecione a ferramenta mais adequada a cada contexto.

O presente artigo tem por objetivo apresentar a fundamentação teórica necessária à elaboração de um POP para acesso, extração e análise de dados no contexto da gestão da produção. A estrutura compreende: (i) conceitos de SIG e arquitetura de software; (ii) fundamentos amplos de bancos de dados, incluindo histórico, paradigmas relacionais e NoSQL, normalização e tipos de relacionamentos; e (iii) os procedimentos de conexão aos dois SGBDs candidatos para a parte prática deste trabalho: PostgreSQL e MongoDB, indicados pelo professor responsável pela disciplina.

2. SISTEMAS DE INFORMAÇÃO GERENCIAL

2.1 Conceito e Componentes

Um Sistema de Informação (SI) é definido como um conjunto organizado de pessoas, processos e tecnologia que coleta, processa, armazena e distribui informações para apoiar a tomada de decisão organizacional (LAUDON; LAUDON, 2014). Stair e Reynolds (2015) acrescentam que um SI eficaz transforma dados brutos em informação útil para ação, completando o ciclo dado → informação → conhecimento → decisão.

Os componentes fundamentais de um SI compreendem: (a) hardware, composto por servidores, sensores e dispositivos de entrada e saída; (b) software, incluindo sistemas ERP, MES, BI e aplicações específicas; (c) banco de dados, o repositório estruturado das informações organizacionais; (d) redes, a infraestrutura de comunicação; e (e) pessoas, os usuários que alimentam, interpretam e decidem com base nas saídas do sistema (O'BRIEN, 2010).

2.2 A Dimensão Sociotécnica

Laudon e Laudon (2014) destacam que um SI opera em um ecossistema composto por três dimensões interdependentes: tecnologia, pessoas e processos organizacionais. Investir exclusivamente na dimensão tecnológica, sem preparar adequadamente as pessoas e redesenhar os processos, é a principal causa de falhas em implantações de sistemas. Na prática industrial, sistemas MES de alto custo são subutilizados porque operadores não foram treinados para registrar paradas corretamente (OLIVEIRA, 2018).

2.3 O Ciclo Dado, Informação e Conhecimento

A pirâmide dado, informação e conhecimento é central para compreender o valor dos SIGs. O dado constitui um fato bruto, sem contexto (ex.: "temperatura: 82°C"); a informação resulta do processamento do dado com atribuição de significado (ex.: "forno acima da temperatura ideal de 75°C"); o conhecimento emerge da combinação de informação com experiência e contexto para orientar a ação (ex.: "reduzir gás em 10%, padrão observado anteriormente") (STAIR; REYNOLDS, 2015). O SIG acelera esse ciclo, reduzindo o tempo de resposta de horas para segundos.

2.4 SIGs na Engenharia de Produção

O Engenheiro de Produção interage com diferentes tipos de sistemas ao longo da cadeia produtiva. O Quadro 1 apresenta os principais SIGs utilizados no contexto industrial.

Quadro 1: Principais Sistemas de Informação na Indústria

Sistema	Função Principal	Usuário Típico
ERP	Integra todos os processos da empresa	Todos os departamentos
MES	Controla a execução da produção em tempo real	Supervisores de produção
WMS	Gerencia movimentação e estoque no armazém	Logística
BI/Analytics	Analisa dados históricos para tendências	Gerência e diretoria
SCADA	Monitora e controla processos industriais	Operação e manutenção

Fonte: adaptado de Laudon e Laudon (2014).

3. ARQUITETURA DE SOFTWARE

3.1 Arquitetura em Camadas

A arquitetura de software define a estrutura organizacional de um sistema, estabelecendo como seus componentes se relacionam e comunicam. A arquitetura em camadas (n-tier) é o modelo predominante em sistemas de informação corporativos e divide o software em responsabilidades bem delimitadas (PRESSMAN; MAXIM, 2016).

Em sua forma clássica de três camadas, a arquitetura compreende: (a) Camada de Apresentação, responsável pela interface com o usuário, exibindo relatórios, dashboards e formulários; (b) Camada de Negócio, que contém as regras do domínio, cálculos, validações e fluxos de processo; e (c) Camada de Dados, que gerencia o armazenamento persistente, sendo o banco de dados o componente central desta

camada. Essa separação traz benefícios de manutenibilidade, escalabilidade e testabilidade independente (SOMMERVILLE, 2019).

3.2 O Banco de Dados como Alicerce do SIG

A qualidade das informações produzidas por um SIG é diretamente dependente da qualidade do modelo de dados subjacente. Um modelo mal projetado gera redundâncias, inconsistências e dificuldades de manutenção que comprometem a confiabilidade das análises. Por isso, a escolha do paradigma de banco de dados, seja relacional ou NoSQL, e a correta modelagem dos dados são decisões arquiteturais de alto impacto (DATE, 2004).

4. BANCOS DE DADOS

4.1 Histórico e Evolução

A história dos bancos de dados acompanha a evolução da computação e reflete a crescente necessidade das organizações de gerenciar grandes volumes de informação de forma estruturada e confiável (ELMASRI; NAVATHE, 2016).

Na década de 1960, os primeiros sistemas de gerenciamento de dados utilizavam modelos hierárquicos e em rede. O modelo hierárquico, adotado pelo IMS da IBM (1966), organizava os dados em estruturas de árvore com relacionamentos pai-filho. O modelo em rede, formalizado pelo CODASYL (1969), permitia relacionamentos mais complexos entre registros, mas exigia do programador conhecimento profundo da estrutura física dos dados (DATE, 2004).

O avanço decisivo veio em 1970, quando Edgar Frank Codd publicou o artigo seminal "A Relational Model of Data for Large Shared Data Banks", propondo o modelo relacional baseado na teoria matemática dos conjuntos e na álgebra relacional. Codd formalizou os conceitos de relação (tabela), atributo (coluna) e tupla (linha), abstraindo o programador da estrutura física de armazenamento (DATE, 2004).

Ao longo das décadas de 1970 e 1980, surgiram os primeiros SGBDs relacionais comerciais: System R (IBM, 1974), Oracle (1979) e DB2 (IBM, 1983). A linguagem SQL foi padronizada pela ISO em 1987, tornando-se a linguagem universal de consulta. Na década de 1990, emergiram SGBDs de código aberto como MySQL (1995) e PostgreSQL (1996), democratizando o acesso à tecnologia (ELMASRI; NAVATHE, 2016).

A partir dos anos 2000, o crescimento exponencial dos dados gerados pela internet, caracterizado por volume, velocidade e variedade (os "3 Vs" do Big Data), evidenciou limitações dos bancos relacionais para certos casos de uso. Surgiram então os bancos NoSQL (Not Only SQL), projetados para escalabilidade horizontal e flexibilidade de esquema. Sistemas como Cassandra (Facebook, 2008), MongoDB (2009) e Redis (2009) ganharam ampla adoção em aplicações web de grande escala (SADALAGE; FOWLER, 2013).

4.2 Bancos de Dados Relacionais

O modelo relacional organiza os dados em tabelas bidimensionais compostas por linhas (tuplas) e colunas (atributos). Cada tabela possui uma chave primária (PK) que identifica univocamente cada registro. Chaves estrangeiras (FK) estabelecem vínculos entre tabelas, implementando os relacionamentos do modelo conceitual (DATE, 2004).

Os SGBDs relacionais garantem as propriedades ACID: Atomicidade (a transação é executada por completo ou desfeita integralmente), Consistência (o banco permanece em estado válido antes e após a transação), Isolamento (transações concorrentes não interferem entre si) e Durabilidade (dados

confirmados persistem mesmo após falhas). Essas propriedades tornam os bancos relacionais ideais para aplicações que exigem integridade rigorosa dos dados, como sistemas financeiros, ERPs e gestão de produção (ELMASRI; NAVATHE, 2016).

Entre os principais SGBDs relacionais disponíveis destacam-se: PostgreSQL, reconhecido pela conformidade com o padrão SQL e extensibilidade; MySQL, amplamente utilizado em aplicações web; Oracle Database, referência em ambientes corporativos de missão crítica; Microsoft SQL Server, integrado ao ecossistema Microsoft; e MariaDB, fork do MySQL com foco em código aberto (ELMASRI; NAVATHE, 2016).

4.3 Bancos de Dados NoSQL

O termo NoSQL (Not Only SQL) engloba uma família diversa de SGBDs que divergem do modelo relacional em aspectos como esquema de dados, linguagem de consulta e garantias de consistência. O surgimento do NoSQL não representa a substituição dos bancos relacionais, mas a ampliação do repertório tecnológico disponível para diferentes necessidades (SADALAGE; FOWLER, 2013).

Os bancos NoSQL são classificados em quatro principais categorias, cada uma com características e casos de uso distintos, conforme apresentado no Quadro 2.

Quadro 2: Categorias de Bancos de Dados NoSQL

Tipo	Estrutura dos Dados	Exemplos	Casos de Uso Típicos
Documento	Documentos JSON/BSON aninhados	MongoDB, CouchDB	Catálogos, perfis de usuário
Chave-Valor	Pares simples chave → valor	Redis, DynamoDB	Cache, sessões, filas
Colunar	Famílias de colunas	Cassandra, HBase	IoT, séries temporais, logs
Grafo	Nós e arestas com propriedades	Neo4j, Amazon Neptune	Redes sociais, recomendação

Fonte: adaptado de Sadalage e Fowler (2013).

Os bancos NoSQL em geral seguem o teorema CAP (Consistency, Availability, Partition Tolerance), que afirma ser impossível garantir simultaneamente consistência total, disponibilidade total e tolerância a partições de rede. Enquanto os bancos relacionais priorizam consistência e disponibilidade (CP ou CA), muitos bancos NoSQL priorizam disponibilidade e tolerância a partições (AP), adotando o modelo de consistência eventual (SADALAGE; FOWLER, 2013).

4.4 Bancos de Dados NewSQL

Uma terceira categoria emergiu na década de 2010: os bancos NewSQL. Esses sistemas buscam combinar as garantias ACID dos bancos relacionais com a escalabilidade horizontal dos sistemas NoSQL. Exemplos notáveis incluem Google Spanner, CockroachDB e TiDB. São especialmente relevantes em cenários que exigem tanto consistência rigorosa quanto capacidade de lidar com volumes massivos de dados distribuídos globalmente (STONEBRAKER; CATTELL, 2011).

4.5 Comparativo entre Paradigmas

A escolha entre um banco relacional, NoSQL ou NewSQL deve ser orientada pelas características do problema, e não por modismo tecnológico. O Quadro 3 sintetiza os principais critérios de comparação.

Quadro 3: Comparativo entre Paradigmas de Bancos de Dados

Critério	Relacional	NoSQL	NewSQL
Esquema	Fixo (schema-on-write)	Flexível (schema-on-read)	Fixo ou semiflexível
Consistência	ACID	Eventual (BASE)	ACID distribuído
Escalabilidade	Vertical (scale-up)	Horizontal (scale-out)	Horizontal
Linguagem	SQL padrão	APIs proprietárias	SQL
Maturidade	Alta (50+ anos)	Média (15+ anos)	Em crescimento
Ideal para	Transações, ERP, finanças	Big Data, web, IoT	Alta escala + consistência

Fonte: elaborado pelo autor com base em Sadalage e Fowler (2013) e Stonebraker e Cattell (2011).

5. NORMALIZAÇÃO E MODELAGEM RELACIONAL

5.1 Conceito de Normalização

A normalização é o processo de organizar os atributos de um banco de dados relacional para minimizar redundâncias e dependências indesejadas, garantindo integridade e facilidade de manutenção. O processo é conduzido por meio das Formas Normais, propostas originalmente por Codd (1972) e expandidas por pesquisadores subsequentes (DATE, 2004).

5.2 Primeira Forma Normal (1FN)

Uma relação está na 1FN quando todos os seus atributos contêm apenas valores **atômicos** (indivisíveis) e não há grupos repetitivos ou atributos multivalorados. Exemplo de violação: armazenar múltiplos produtos em um único campo "produtos_pedido" separados por vírgula. A solução é decompor o atributo em uma tabela separada, garantindo uma linha por produto. A 1FN é o requisito mínimo para que uma tabela seja considerada uma relação no sentido matemático (DATE, 2004).

5.3 Segunda Forma Normal (2FN)

Uma relação está na 2FN se está na 1FN e todos os atributos não-chave dependem funcionalmente da **chave primária inteira**, e não apenas de parte dela, condição relevante em tabelas com chaves compostas. Exemplo de violação: armazenar o nome do produto em uma tabela de itens de pedido onde a PK é composta por (id_pedido, id_produto). O nome do produto depende apenas de id_produto, não do conjunto. A solução é mover o nome para a tabela de produtos. Eliminar dependências parciais reduz anomalias de atualização (DATE, 2004).

5.4 Terceira Forma Normal (3FN)

Uma relação está na 3FN se está na 2FN e não existem **dependências transitivas** entre atributos não-chave. Dependência transitiva ocorre quando um atributo não-chave depende de outro atributo não-chave, e não diretamente da PK. Exemplo de violação: armazenar cidade e estado do cliente na mesma tabela quando o estado depende da cidade (não da PK cliente). A solução é criar uma tabela de cidades referenciada por FK. Um banco em 3FN garante que cada fato seja armazenado em um único lugar, eliminando anomalias de inserção, atualização e exclusão (DATE, 2004).

5.5 Tipos de Relacionamentos

Os relacionamentos entre entidades são classificados de acordo com a cardinalidade, que expressa quantas ocorrências de uma entidade se associam a quantas ocorrências de outra (ELMASRI; NAVATHE, 2016).

Relacionamento 1:1 (um para um): cada ocorrência da entidade A se relaciona com no máximo uma ocorrência da entidade B, e vice-versa. Exemplo: cada Operador possui um único Crachá, e cada Crachá pertence a um único Operador. Implementado adicionando a FK de um lado na tabela do outro.

Relacionamento 1:N (um para muitos): uma ocorrência da entidade A se relaciona com várias ocorrências da entidade B, mas cada B se relaciona com apenas um A. Exemplo: uma Máquina executa várias Ordens de Produção, mas cada Ordem é executada em uma única Máquina. Implementado com FK na tabela do lado N.

Relacionamento N:M (muitos para muitos): várias ocorrências de A se relacionam com várias ocorrências de B. Exemplo: um Produto pode aparecer em várias Ordens de Produção, e uma Ordem pode conter vários Produtos. Implementado por meio de uma tabela associativa (junção) que contém as FKs de ambas as entidades e pode acrescentar atributos do relacionamento, como quantidade e tempo de processo.

Agregação e Composição: ambas representam relações "todo-parte". Na agregação, as partes podem existir independentemente do todo (ex.: um Componente pode existir sem estar associado a um Produto). Na composição, forma mais forte de agregação, as partes não existem sem o todo (ex.: os Itens de uma Ordem de Produção não têm significado sem a Ordem que os originou) (ELMASRI; NAVATHE, 2016).

6. SGBDS CANDIDATOS E PROCEDIMENTO DE CONEXÃO

6.1 Visão Geral

Para a parte prática deste trabalho, o professor indicou dois SGBDs candidatos: PostgreSQL e MongoDB. O PostgreSQL é um SGBD relacional, que armazena dados em tabelas e utiliza a linguagem SQL para consultas. O MongoDB é um SGBD NoSQL do tipo documento, que armazena dados no formato JSON e utiliza sua própria linguagem de consulta. Ambos possuem ferramentas gráficas gratuitas que permitem acessar um banco de dados existente sem necessidade de programação. O procedimento de conexão para cada um é descrito a seguir.

6.2 Conexão ao PostgreSQL via pgAdmin 4

O pgAdmin 4 é a ferramenta gráfica oficial do PostgreSQL. Para acessar um banco de dados PostgreSQL existente no Windows, o procedimento é:

- Acessar o site pgadmin.org, clicar em Download, selecionar a versão para Windows e instalar seguindo os passos na tela;
- Abrir o pgAdmin 4 pelo menu Iniciar do Windows;
- No painel esquerdo, clicar com o botão direito em Servers e selecionar Register e depois Server;
- Na aba General, preencher o campo Name com um nome para identificar a conexão, como por exemplo Dataset Professor;
- Na aba Connection, preencher os campos Host, Port (geralmente 5432), Maintenance database, Username e Password com as credenciais fornecidas pelo professor;
- Clicar em Save. O banco aparecerá no painel esquerdo. Para executar consultas, clicar com o botão direito no banco e selecionar Query Tool.

6.3 Conexão ao MongoDB via MongoDB Compass

O MongoDB Compass é a ferramenta gráfica oficial do MongoDB. Para acessar um banco de dados MongoDB existente no Windows, o procedimento é:

- Acessar o site mongodb.com/products/compass, baixar a versão para Windows e instalar seguindo os passos na tela;
- Abrir o MongoDB Compass pelo menu Iniciar do Windows;
- Na tela inicial, localizar o campo New Connection. Inserir a string de conexão fornecida pelo professor, que tem o formato: `mongodb://usuario:senha@host:porta/nome_do_banco`;
- Clicar em Connect. O banco aparecerá na tela com suas coleções listadas no painel esquerdo;
- Para explorar os dados, clicar em uma coleção. Para executar consultas, clicar em Documents e utilizar o campo Filter, ou acessar a aba Aggregations para consultas mais avançadas.

REFERÊNCIAS

DATE, C. J. Introdução a Sistemas de Banco de Dados. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, R.; NAVATHE, S. B. Sistemas de Banco de Dados. 7. ed. São Paulo: Pearson, 2016.

LAUDON, K. C.; LAUDON, J. P. Sistemas de Informação Gerenciais. 11. ed. São Paulo: Pearson, 2014.

O'BRIEN, J. A. Sistemas de Informação e as Decisões Gerenciais na Era da Internet. 3. ed. São Paulo: Saraiva, 2010.

OLIVEIRA, D. P. R. Sistemas de Informações Gerenciais. 16. ed. São Paulo: Atlas, 2018.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 Documentation. 2024. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 03 maio 2026.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de Software: uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.

SADALAGE, P. J.; FOWLER, M. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston: Addison-Wesley, 2013.

SOMMERVILLE, I. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

STAIR, R. M.; REYNOLDS, G. W. Princípios de Sistemas de Informação. 11. ed. São Paulo: Cengage, 2015.

STONEBRAKER, M.; CATTELL, R. 10 Rules for Scalable Performance in 'Simple Operation' Datastores. Communications of the ACM, v. 54, n. 6, p. 72-80, 2011.